

Big Data Analysis
University of Minho

Guide 2

Apache Spark - Optimizations and HPC

2025/2026

1 Part 1 - Running Locally

First, you need to connect to the Supercomputer Deucalion.

```
ssh deucalion
```

Now, you need to request a node to create a Python environment and follow the instructions in *Guide 1* for installing Python libraries.

To allocate a node, run the following code:

```
srun --time=00:45:00 --partition=dev-arm --account=<ASSOCIATEDPROJECT>a --pty bash
```

Inside the compute node allocated to you, create the Python environment in a known directory (/projects/<ASSOCIATEDPROJECT>/<USER>/), and in another terminal, you need to make a tunnel to access the Jupyter Notebooks.

```
ssh deucalion -L <PORT>:localhost:<PORT> -L 8080:<ALLOCATEDNODE>:8080  
ssh <ALLOCATEDNODE> -L <PORT>:localhost:<PORT>
```

Inside the allocated node run:

```
module load "Python/3.12.3-GCCcore-13.3.0"  
source env-spark/bin/activate  
jupyter notebook --no-browser --port=<PORT>
```

2 Part 2 - Setting up a Spark Cluster

First, on the login node, in your *projects folder*, download and unzip Apache Spark from <https://spark.apache.org/downloads.html>. Use `wget` and `unzip` for these tasks.

NOTE: Be aware that the workers, master, and client must work within the same Python version. To ensure this happens, you must update the `spark-config.sh` file accordingly. As such, add the following lines in this file: `$SPARK_HOME/sbin/spark-config.sh`:

```
source /share/env/module_select.sh  
module purge  
module load "Python/3.12.3-GCCcore-13.3.0"  
module load "Java/17.0.6"  
module load "OpenMPI/5.0.3-GCC-13.3.0"  
source /projects/PATH/T0/ENV/bin/activate
```

For submitting a job, you should create a bash script, which will be given to the sbatch command.

```
#!/bin/bash
#SBATCH --job-name=spark-cluster      # create a short name for your job
#SBATCH --nodes=3                    # node count
#SBATCH --cpus-per-task=48           # cpu-cores per task (>1 if multi-threaded tasks)
#SBATCH --mem=28G                    # memory per node
#SBATCH --time=00:55:00              # total run time limit (HH:MM:SS)
#SBATCH --account=<PROJECTACCOUNTASSOCIATED> #add an a in the end for the arm nodes
#SBATCH --partition=normal-arm

source /share/env/module_select.sh
module purge
module load "Python/3.12.3-GCCcore-13.3.0"
module load "Java/17.0.6"
module load "OpenMPI/5.0.3-GCC-13.3.0"
source /projects/PATH/TO/ENV/bin/activate
export PYTHONPATH=/projects/PATH/TO/ENV/bin/python
export PYSARK_PYTHON=/projects/PATH/TO/ENV/bin/python
export PYSARK_DRIVER_PYTHON=/projects/PATH/TO/ENV/bin/python
export SPARK_HOME=/projects/PATH/TO/SPARK/spark-3.5.4-bin-hadoop3
export PATH="${SPARK_HOME}/bin:${PATH}"

# Get all worker nodes and write to the config file of Spark
scontrol show hostnames | sed -n '2,$p' > $SPARK_HOME/conf/workers

# Start the spark cluster
$SPARK_HOME/sbin/start-all.sh
sleep infinity
```

3 Slow Querying

Now, take the code from the previous class and update the SparkSession without specifying your master, since it will be submitted via your bash script. Run the following query and analyze where Spark struggles using explain("extended").

```
from pyspark.sql.functions import col, count, explode, split

df = spark.read.option("sep", "\t").option("header", "true").csv("title.basics.tsv.gz")
akas_df = spark.read.option("sep", "\t").option("header", "true").csv("title.akas.tsv.gz")

df = df.withColumn("genres", explode(split(col("genres"), ",")))

joined_df = df.join(akas_df, df.titleId == akas_df.titleId, "inner")

slow_query = joined_df.filter(col("startYear") >= 2000) \
    .groupBy("startYear", "genres") \
    .agg(
        count("primaryTitle").alias("title_count"), # Count per genre per year
        count("region").alias("region_count") # Count how many regions have the title
    ) \
    .orderBy(col("title_count").desc(), col("region_count").desc())
```

```
slow_query.explain("extended")
slow_query.show()
```

Tasks

Reduce execution time by minimizing shuffling, leveraging partitioning, caching, and optimizing joins.

Performance Bottlenecks:

1. What is the biggest performance bottleneck in the initial query?
2. Identify the main factors affecting performance (e.g., shuffling, skewed data, large joins, partitioning issues).
3. How does the physical plan (explain("extended")) reflect these bottlenecks?
4. How does the Spark UI (DAG, Stage Breakdown, and Executors Tab) help diagnose these issues?

Optimization Comparison:

1. Create a comparison table of different optimization techniques (e.g., broadcast joins, partitioning, caching, coalesce/repartition).
2. For each optimization, measure execution time, number of shuffle stages, number of partitions, and memory usage.
3. How do different optimizations affect CPU and memory usage?

Combining Optimizations:

1. Apply multiple optimizations together (e.g., broadcast joins + caching + repartitioning).
2. How do different combinations affect performance?
3. Is there an optimal combination that provides the best speedup?
4. What happens when you over-optimize? Does it cause performance degradation?

Measuring and Interpreting Improvements:

1. Which optimization led to the biggest speed improvement?
2. What metrics show the biggest difference (e.g., task duration, number of shuffle writes, GC time)?
3. How do optimizations affect scalability when running on a larger dataset?
4. What trade-offs exist between different optimizations (e.g., reducing shuffle vs. increasing memory usage)?

4 Learning Outcomes

Understand Apache Spark optimizations and their performance impact. Learn how to setup and launch a Apache Spark cluster in HPC environments.